

Service Center iOS App Documentation

Features Overview

[High-level summary of the app's main features]

Team Contributions:

Written by Robert Agnvall

Sprint 1 Documentation:

Design an Interface for Creating Posts

Overview

This feature introduces the design and implementation of an interface for creating posts within the Service Center application. It includes UI components for displaying posts, structured data handling for fetching and displaying posts, and necessary extensions for formatting post metrics. The implementation utilizes SwiftUI to create a modular and responsive UI.

Changes Introduced

This PR adds the following components:

1. PostCard Component

- File: `Models/PostCard/PostCard.swift`
- Description: The `PostCard` view acts as a container for displaying individual posts.
- Components Used:
 - `PostCardHeader` - Displays user profile details.
 - `PostCardBody` - Displays post image, interactions, and description.
- Properties:
 - `profile_img: String` - URL or name of the profile image.
 - `profile_name: String` - Name of the user.
 - `profile_id: String` - Identifier for the profile.
 - `image: String` - Image associated with the post.
 - `like_count: Int` - Number of likes.
 - `comment_count: Int` - Number of comments.

- `view_count: Int` - Number of views.
- `description: String` - Text content of the post.

2. PostCardBody Component

- File: `Models/PostCard/PostCardBody.swift`
- Description: The `PostCardBody` component displays the post image and engagement metrics.
- Features:
 - Uses `Image(post_image_name).resizable()` for dynamic image rendering.
 - Rounded corners using a custom extension.
 - Displays interactive elements: likes, comments, views, and a bookmark icon.
- Formatted Number Display:
 - Uses an extension function `formattedString()` for better readability of large numbers.

3. PostCardHeader Component

- File: `Models/PostCard/PostCardHeader.swift`
- Description: Displays user profile information and an options button.
- Features:
 - Profile picture with rounded rectangle styling.
 - Profile name and ID with proper alignment.
 - Uses `HStack` for structured layout.

4. Fetching and Handling JSON Data

- File: `Shared/FetchDataJson.swift`
- Description: Implements a data model and fetcher for reading post data from a JSON file.
- Key Elements:
 - `Post` struct for defining a post's properties (conforming to `Codable` and `Identifiable`).
 - `ReadJsonData` class for loading JSON data into an observable object.
 - Uses `Bundle.main.url(forResource: "Posts", withExtension: "json")` to locate the JSON file.

5. Post List View

- File: `Shared/PostListView.swift`

- Description: Displays a list of **PostCard** components populated from JSON data.
- Implementation:
 - Uses **@ObservedObject var postData = ReadJsonData()** to fetch and manage post data.
 - Iterates through **postData.posts** using **List {}**.

6. Assets for Profile and Post Images

- Files: Located in **Assets.xcassets/Post_img/** and **Assets.xcassets/Profile_img/**
- Description:
 - Image assets structured under **.imageset/** for different scale resolutions.
 - Supports **1x**, **2x**, and **3x** resolutions.

7. Extensions for SwiftUI View and Integer Formatting

- File: **Shared/Extensions.swift**
- New Extensions:
 - **roundedCorner(_ radius: CGFloat, corners: UIRectCorner):**
Clips the view with specific rounded corners.
 - **formattedString():** Converts large numbers into a readable format (e.g., **10K** or **1.2M**).

Project File Changes

- **Service Center.xcodeproj/project.pbxproj** updated to include:
 - New files in **Sources**.
 - Image assets in **Resources**.
 - Grouping structure updates.

Testing and Validation

- Manual Testing:
 - Verified UI layout and responsiveness.
 - Confirmed data loading from JSON.
 - Checked interaction elements (likes, comments, views).
- Preview Validation:
 - **PostListView_Previews** provides a navigation-wrapped preview.
- Debugging Hooks:
 - Prints error messages if JSON file is missing.

Sprint 2 Documentation:

Save Posts Feature

Overview

This pull request introduces a new feature allowing users to create and save posts in the Service Center application. The implementation includes a UI for post creation, integration with Firebase for image uploading and post storage, and an updated post list that retrieves saved posts dynamically. The system now supports real-time fetching and displaying of posts, replacing the previous JSON-based static implementation.

Changes Introduced

1. CreatePostView - UI for Creating Posts

- **File:** `Models/CreatePostView.swift`
- **Description:**
 - Provides an interface for users to create new posts.
 - Allows image selection using `PHPickerView`.
 - Stores descriptions and uploads images to Firebase.
- **Key Features:**
 - `@StateObject private var firebaseManager = FirebaseManager.shared` - Integrates Firebase.
 - Uses a `Button` to open the image picker.
 - Dismisses after successfully creating a post.

2. PostCardData - Data Model for Posts

- **File:** `Models/PostCard/PostCardData.swift`
- **Description:**
 - Defines the structure of a post for consistent data handling.
- **Properties:**
 - `profile_img, profile_name, profile_id, image, like_count, comment_count, view_count, description`
 - Implements `Identifiable` for SwiftUI list integration.

3. PostCard - UI for Displaying Posts

- **File:** `Models/PostCard/PostCard.swift`
- **Description:**
 - Represents a single post using `PostCardData`.
 - Now dynamically loads images from Firebase.
- **Enhancements:**
 - Uses `AsyncImage(url: URL(string: post_image_name))` for Firebase image support.
 - Improved UI with background styling and shadow effects.

4. PostCardHeader - Updated UI for Profile Display

- **File:** `Models/PostCard/PostCardHeader.swift`
- **Description:**
 - Displays user profile images dynamically from Firebase.
- **Enhancements:**
 - Uses `AsyncImage(url: URL(string: Profile_img))` to load images.

5. PostCardBody - Updated Post UI

- **File:** `Models/PostCard/PostCardBody.swift`
- **Description:**
 - Displays post images, descriptions, and interaction metrics.
- **Enhancements:**
 - Supports dynamic image loading from Firebase.
 - Improved UI with `AsyncImage`, colored description section, and interaction bar.

6. PostListView - Dynamic Post Fetching

- **File:** `Shared/PostListView.swift`
- **Description:**
 - Displays all saved posts from Firebase.
- **Enhancements:**
 - Uses `FirestoreManager.shared.posts` instead of static JSON data.
 - Supports pull-to-refresh (`.refreshable { firestoreManager.loadPosts() }`).
 - Includes a button to open `CreatePostView`.

7. FirestoreManager - Firestore Integration

- **File:** `Shared/FireBaseManager.swift`

- **Description:**
 - Handles Firebase Firestore operations for storing and retrieving posts.
- **Key Methods:**
 - `uploadImage(_ image: UIImage, completion: @escaping (String?) -> Void)` - Uploads images to Firebase Storage.
 - `savePost(_ post: PostCardData)` - Saves post data to Firestore.
 - `loadPosts()` - Fetches posts in real-time and updates UI.

8. Project Configuration Changes

- **Service Center.xcodeproj/project.pbxproj** updated to include:
 - Firebase dependencies (`FirebaseStorage`, `FirebaseFirestore`).
 - New files related to post creation and management.

Testing and Validation

- **Manual Testing:**
 - Created and saved multiple posts with images.
 - Verified successful uploads to Firebase.
 - Ensured posts appear dynamically in `PostListView`.
- **Preview Validation:**
 - `PostCard_Previews` used to validate UI rendering.
- **Debugging Hooks:**
 - Prints messages on image upload progress.
 - Logs errors if Firebase operations fail.

Summary

This PR successfully transitions post handling from static JSON to dynamic Firebase storage. It introduces a user-friendly post creation UI, real-time post fetching, and Firebase-based image handling.

Sprint 3 Documentation:

Messaging System Documentation

Introduction

The **Messaging System** in *Service Center* provides users with a seamless, real-time communication platform. Built using **Firebase Firestore** and **Firebase Storage**, it supports direct messaging, image sharing, and message previews. The system ensures efficient data retrieval and smooth user interaction, enabling meaningful conversations between users.

This document provides an in-depth look at the **messaging system's features, architecture, and functionality**, outlining how messages are structured, processed, and displayed within the app.

Features

1. Real-Time Messaging

- Users can send and receive messages instantly using Firebase Firestore.
- Conversations are dynamically updated without requiring manual refreshes.

2. Message Previews & Chat List

- Users can view recent conversations in a chat list, displaying the latest message.
- Only the **10 most recent** chat previews are shown, sorted by the latest activity.
- Unread messages are visually indicated.

3. Image Attachments

- Users can upload and send images directly within a conversation.
- Images are stored in **Firebase Storage**, and URLs are saved in Firestore.
- Multiple image uploads are supported, with previews before sending.

4. User Search & Starting New Chats

- Users can **search for other users** to start new conversations.
- Recently messaged users are prioritized in search results.
- Selecting a user navigates to a **new chat thread**.

5. Conversation Management

- Users can **delete** entire conversations from their chat list.
 - Messages are stored efficiently to support long-term access.
-

Architecture

1. Firebase Firestore (Database Layer)

- Messages are stored in a **"chats"** collection.
- Each conversation has a unique document ID (`senderId_receiverId`).
- A **"messages"** subcollection stores individual messages.

2. Firebase Storage (Image Handling)

- Images are uploaded and stored in **"chat_images/"**.
- Image URLs are retrieved and linked within messages for easy access.

3. ViewModel & Data Binding

- `MessagesViewModel` manages real-time updates and data processing.
 - Firestore snapshot listeners enable automatic UI updates.
 - Efficient querying ensures that messages and previews load quickly.
-

Key Components

1. Message Structure

Each message contains:

- **Sender ID**
- **Receiver ID**
- **Message text** (if applicable)
- **Timestamp**
- **Image URL** (if an image is attached)

Messages are dynamically decoded and displayed in the user interface.

2. Chat Interface

- Messages appear in a chat bubble format, distinguishing sender and receiver.
- Timestamps group messages by day, reducing clutter.
- Messages are automatically **scrolled into view** upon receipt.

3. New Message Flow

- The **New Message** view allows users to search for existing users.
- Search results are updated **with a debounce mechanism** for efficiency.
- Selecting a user initiates a new conversation thread.

4. Image Uploading

- Users can select images from their device gallery.
- Images are uploaded asynchronously with real-time progress indicators.
- Successfully uploaded images appear **inline within the chat**.

5. Message Previews

- The home screen displays chat previews, showing:
 - The **latest message**
 - **Timestamp of the last activity**
 - **Read/unread status**
 - Conversations are **sorted dynamically** based on recent activity.
-

Future Enhancements

To improve the messaging experience, the following features could be implemented:

- **Typing Indicators** – Show when the other user is typing.
 - **Message Reactions** – Enable users to react with emojis.
 - **Read Receipts** – Indicate when messages have been read.
 - **Group Messaging** – Expand support for multi-user conversations.
-

Conclusion

The **Messaging System** in *Service Center* provides users with a robust and intuitive way to communicate. By leveraging **Firebase Firestore** and **Firebase Storage**, the system ensures real-time updates, efficient data handling, and seamless integration of text and image messages.

With its structured architecture and smooth user experience, the messaging feature plays a crucial role in enhancing user engagement within the *Service Center* platform. Future enhancements can further refine its functionality, making communication even more interactive and dynamic.

Sprint 4 Documentation:

Overview

The feature introduces several new features, modifications, and bug fixes to improve the *Service Center* application. Key changes include new views, improved user authentication, enhanced post management, Firestore data defaults, and an updated messaging system.

Features Added

1. RootView (New Entry Point for App)

- Introduces a new `RootView.swift` file, responsible for handling authentication state and rendering appropriate views.
- Uses `AuthManager` to determine whether to show `ContentView`, `LoginView`, or a `LoadingView`.
- Ensures a smooth user experience by preventing unnecessary reloads.

2. FirestoreDefaults (Ensuring Data Integrity)

- `FirestoreDefaults.swift` injects default values into Firestore data dictionaries before decoding.
- Prevents crashes due to missing values by assigning defaults for:
 - `documentId`
 - `jobRequests` (empty array)
 - `isLikedByCurrentUser` (false)
 - `tags` (empty array)
 - `like_count`, `comment_count`, and `view_count` (set to 0 if missing).
- Ensures uniform data across all Firestore documents.

3. Updated Authentication System

- `AuthManager` now loads user data more robustly:
 - Ensures `uid` is always set.
 - Provides default profile pictures and user metadata if missing.
 - Automatically updates Firestore if required fields are missing.
 - Improved error handling and logging.

4. Post Management Improvements

- `FireBasePostManager.swift`:
 - Now updates `postsCreated` count more reliably.
 - Ensures decrementing `postsCreated` only occurs if the field is present.
 - More robust logging for debugging user post deletions.
- `PostCardBody.swift`:
 - Now dynamically adjusts image scaling to improve UI.
- `PostCardData.swift`:
 - Now conforms to `Codable` to simplify Firestore decoding.

5. Enhanced Messaging System

- `MessagesViewModel.swift`:
 - Prevents messages from being sent with an empty `chatId`.

- **ChatView.swift:**
 - Displays "No messages yet" when no conversation exists.
 - Improved username handling—Firestore fallback only applies if no username was provided.
 - Fetches profile pictures more efficiently.
 - Improved error logging and debugging messages.
- **MessageBubble.swift:**
 - Now supports image-based messages in addition to text.

6. New Comments System

- **Models:**
 - **commentDataModel.swift:** Defines the **Comment** structure for handling threaded discussions.
- **Views:**
 - **CommentRowView.swift:** Handles displaying individual comments and replies.
 - **CommentsView.swift:** Manages full comment threads with reply functionality.
- **ViewModels:**
 - **CommentsViewModel.swift:** Handles real-time Firestore updates, comment sending, and deletion.

Files Modified

Deleted Files

- **TestView.swift**
- **UserPostsView.swift**
- **UsernameInputView.swift**
- **HorizontalPostView.swift**

Renamed Files

- **FetchDataJson.swift** → **Alan-Features/FetchDataJson.swift**

Bug Fixes & Code Refactoring

- **Review System Fixes:**
 - **ReviewCardView.swift** now correctly fetches the reviewer's username instead of the review target.

- **Job Request Handling Fixes:**
 - `RequestsView.swift` now properly updates `jobStatus` when a request is accepted/rejected.
- **Improved AsyncImage Handling:**
 - `PostCardBody.swift` now ensures proper image scaling and error handling.

Summary

The feature significantly enhances the Service Center application by introducing a new authentication entry point, improving data consistency in Firestore, refining the post and messaging systems, and adding a structured commenting system. These updates collectively contribute to better stability, a cleaner UI, and a more user-friendly experience.

Next Steps

- **Further UI enhancements:** Improve styling for `CommentsView.swift` and `ChatView.swift`.
- **Testing & Bug Fixing:** Validate the new authentication handling and Firestore data defaults.
- **Performance Optimization:** Optimize Firestore queries in `CommentsViewModel.swift` and `MessagesViewModel.swift`.

Sprint 3 Documentation

Overview

The `FirestoreSavePosts` class is responsible for managing post saving functionality, allowing users to save, unsave, and retrieve posts they are interested in. It provides a complete interface for interacting with the Firestore database specifically for saved post operations.

Components

1. `FirestoreSavePosts`

A singleton class that manages the saving and retrieval of posts for users.

Variables:

- `shared`: Static instance of `FirestoreSavePosts` following singleton pattern
- `db`: Private instance of Firestore database for database operations

Implemented Functions:

`getUserDocumentId(forUsername: String, completion: @escaping (String?) -> Void)`

- **Purpose:** Retrieves the user's document ID from Firestore based on their username
- **Parameters:**
 - `username`: The username to query in the users collection
 - `completion`: Callback returning the document ID or nil if not found
- **Key Challenges:**
 - Handling asynchronous Firestore queries
 - Proper error handling when a user is not found

`savePost(postId: String, userId: String, completion: @escaping (Bool) -> Void)`

- **Purpose:** Saves a post to a user's saved posts collection
- **Parameters:**
 1. `postId`: The ID of the post to save
 2. `userId`: The username of the user saving the post
 3. `completion`: Callback returning success (true) or failure (false)

- **Process:**
 1. First retrieves the user's document ID using the helper function
 2. Creates a savedPost document with the postId and timestamp
 3. Writes the document to the user's savedPosts subcollection

unsavePost(postId: String, userId: String, completion: @escaping (Bool) -> Void)

- **Purpose:** Removes a post from a user's saved posts collection
- **Parameters:**
 1. **postId:** The ID of the post to unsave
 2. **userId:** The username of the user unsaving the post
 3. **completion:** Callback returning success (true) or failure (false)
- **Process:**
 1. Retrieves the user's document ID
 2. Deletes the specified post document from the user's savedPosts subcollection

isPostSaved(postId: String, userId: String, completion: @escaping (Bool) -> Void)

- **Purpose:** Checks if a post is saved by a specific user
- **Parameters:**
 1. **postId:** The ID of the post to check
 2. **userId:** The username of the user
 3. **completion:** Callback returning whether the post is saved (true) or not (false)
- **Process:**
 1. Retrieves the user's document ID
 2. Checks if the document exists in the user's savedPosts subcollection

getSavedPostIds(userId: String, completion: @escaping ([String]) -> Void)

- **Purpose:** Retrieves all saved post IDs for a user
- **Parameters:**
 1. **userId:** The username of the user
 2. **completion:** Callback returning an array of saved post IDs
- **Process:**
 1. Retrieves the user's document ID
 2. Calls fetchSavedPostIds to get the actual post IDs

fetchSavedPostIds(userDocId: String, completion: @escaping ([String]) -> Void)

- **Purpose:** Helper function to retrieve saved post IDs from Firestore
- **Parameters:**
 1. **userDocId:** The document ID of the user

- 2. **completion**: Callback returning an array of saved post IDs
- **Process**:
 1. Queries the savedPosts subcollection
 2. Orders results by savedAt timestamp (descending)
 3. Extracts and returns postId values

getSavedPosts(userId: String, completion: @escaping ([PostCardData]) -> Void)

- **Purpose**: Retrieves complete post data for all saved posts
- **Parameters**:
 1. **userId**: The username of the user
 2. **completion**: Callback returning an array of PostCardData objects
- **Process**:
 1. Gets the user document ID
 2. Gets the saved post IDs
 3. Fetches the complete post data for each ID

fetchPostsFromIds(postIds: [String], completion: @escaping ([PostCardData]) -> Void)

- **Purpose**: Fetches complete post data for an array of post IDs
- **Parameters**:
 1. **postIds**: Array of post IDs to fetch
 2. **completion**: Callback returning an array of PostCardData objects
- **Process**:
 1. Uses DispatchGroup to manage multiple asynchronous requests
 2. Calls fetchSinglePost for each ID
 3. Sorts the results to match the original order of post IDs

sortPostsInOrder(posts: [PostCardData], postIds: [String]) -> [PostCardData]

- **Purpose**: Ensures posts are returned in the same order as the IDs were provided
- **Parameters**:
 - **posts**: Array of PostCardData objects
 - **postIds**: Array of post IDs in the desired order
- **Returns**: Sorted array of PostCardData objects

fetchSinglePost(dbRef: Firestore, postId: String, completion: @escaping (PostCardData?) -> Void)

- **Purpose**: Fetches a single post's data from Firestore
- **Parameters**:
 1. **dbRef**: Firestore database reference
 2. **postId**: ID of the post to fetch

3. **completion**: Callback returning a PostCardData object or nil if not found
- **Process**:
 1. Queries the post document
 2. Safely extracts all fields with default values
 3. Creates and returns a PostCardData object

2. SavedPostsSheetView

A SwiftUI view that displays saved posts in a scrollable list format.

Variables:

- **authViewModel**: EnvironmentObject that provides access to the current user information
- **savedPosts**: State variable that stores the array of saved PostCardData
- **isLoading**: State variable that tracks the loading state of posts

Implemented Functions:

loadSavedPosts()

- **Purpose**: Fetches all saved posts for the current user
- **Process**:
 - Retrieves the current user from authViewModel
 - Calls FirebaseSavePosts.shared.getSavedPosts with the user's username
 - Updates the savedPosts array with the retrieved posts
 - Sets isLoading to false when complete
- **Key Challenges**:
 - Handling asynchronous loading states
 - Proper error handling if user data is not available

View Structure:

- **Loading State**: Displays a centered ProgressView when posts are being loaded
- **Empty State**: Displays a "No saved posts" message with a bookmark slash icon when there are no saved posts
- **Content State**: Displays a scrollable list of PostCard components when saved posts exist
 - Each PostCard is populated with the corresponding PostCardData
 - Cards are arranged in a LazyVStack for efficient rendering

2. Bookmark Functionality (from DetailedPostView)

Functions:

checkIfPostIsSaved()

- **Purpose:** Checks if the current post is saved by the current user
- **Process:**
 1. Retrieves the current user from authViewModel
 2. Calls `FirestoreSavePosts.shared.isPostSaved` with the post ID and user's username
 3. Updates the `isPostSaved` state variable based on the result

toggleBookmark(currentUser: User)

- **Purpose:** Toggles the bookmark status of a post (save/unsave)
- **Parameters:**
 - `currentUser`: The current logged-in user
- **Process:**
 - Sets `isSavingPost` to true to manage loading state
 - If the post is already saved:
 - Calls `FirestoreSavePosts.shared.unsavePost` to remove the bookmark
 - Updates `isPostSaved` to false on success
 - If the post is not saved:
 - Calls `FirestoreSavePosts.shared.savePost` to add the bookmark
 - Updates `isPostSaved` to true on success
 - Sets `isSavingPost` to false when operation completes
- **Key Challenges:**
 - Managing loading state during asynchronous operations
 - Providing visual feedback for bookmark actions

Next Steps

- Add search functionality within saved posts
- Implement sorting options (by date saved, alphabetically, etc.)
- Optimize with pagination for users with many saved posts
- Add categories or tagging system for saved posts

Sprint 4 Documentation

Overview

The `CreatePostView` component allows users to create and publish new service posts. It integrates with `FirestoreManager` to handle image uploads, post creation, tag management, and user account updates. This documentation covers the interaction between `CreatePostView` and the corresponding functions in `FirestoreManager`.

Components

1. CreatePostView

A SwiftUI view that provides the UI for creating new service posts.

State Variables:

- `firebaseManager`: Reference to the shared `FirebaseManager` instance
- `hashtagViewModel`: Manages hashtag selection
- `authManager`: Manages authentication
- `locationManager`: Manages location selection and data
- `selectedImage`: The image chosen for the post
- `title`: Post title
- `description`: Post description
- `price`: Service price
- `timeUnit`: Time unit for the price (hour, day, service)
- `isSubmitted`: Tracks if submission was attempted for validation

Implemented Functions:

`resetFields()`

- **Purpose:** Resets all form fields to their default values
- **Process:**
 1. Clears selected image
 2. Resets text fields
 3. Removes all selected hashtags

`createPost()`

- **Purpose:** Creates and saves a new post to Firebase
- **Process:**
 1. Validates required fields (title, description, image)
 2. Uploads the selected image using `firebaseManager.uploadImage()`
 3. Creates a `PostCardData` object with all post information
 4. Saves the post using `firebaseManager.savePost()`
 5. Increments the user's postsCreated count
 6. Updates unique tags in Firebase
 7. Navigates back to the home tab upon success

`incrementPostsCreatedForCurrentUser()`

- **Purpose:** Increments the postsCreated field for the current user
- **Process:**
 1. Gets the current user
 2. Uses Firestore to increment the postsCreated field in the user document

View Lifecycle Events:

- **onAppear:**
 1. Resets fields and validation state
 2. Updates unique tags with default hashtags
 3. Fetches existing unique tags for the hashtag selection menu

2. FirebaseManager Functions Used by CreatePostView

uploadImage(_ image: UIImage, completion: @escaping (String?) -> Void)

- **Purpose:** Uploads an image to Firebase Storage
- **Parameters:**
 - **image:** The UIImage to upload
 - **completion:** Callback returning the download URL as a String
- **Process:**
 - Converts the image to JPEG data
 - Creates a unique filename
 - Sets content type metadata
 - Uploads the image data to Firebase Storage
 - Retrieves and returns the download URL
- **Key Challenges:**
 - Handling compression and data conversion
 - Error handling during the upload process

savePost(_ post: PostCardData)

- **Purpose:** Saves a post document to Firestore
- **Parameters:**
 - **post:** PostCardData object containing all post information
- **Process:**
 - Creates a dictionary with all post data fields
 - Adds optional fields like price and timeUnit if present
 - Adds any multiple image URLs if available
 - Adds the post to the "posts" collection in Firestore
 - Reloads posts after successful save
- **Key Challenges:**
 - Ensuring all fields are properly formatted for Firestore storage
 - Handling optional fields appropriately

updateUniqueTags(newTags: [String], completion: @escaping (Bool) -> Void)

- **Purpose:** Updates the collection of unique hashtags stored in Firebase
- **Parameters:**
 - **newTags:** Array of hashtags to add to the unique tags collection
 - **completion:** Callback returning success/failure
- **Process:**
 - Retrieves the current unique tags document
 - Merges existing tags with new tags, removing duplicates
 - Updates the uniqueTags document with the combined array
 - Creates the document if it doesn't exist
- **Key Challenges:**
 - Ensuring no duplicates exist in the tags collection
 - Handling document creation vs. update logic

fetchUniqueTags(completion: @escaping ([String]) -> Void)

- **Purpose:** Retrieves all unique hashtags from Firebase
- **Parameters:**
 - **completion:** Callback returning the array of unique tags
- **Process:**
 - Fetches the uniqueTags document from metadata collection
 - Extracts the tags array
 - Returns empty array if document doesn't exist or on error
- **Key Challenges:**
 - Error handling if the document doesn't exist

updateUserPostsCreated(uid: String, postsCreated: Int, completion: @escaping (Bool) -> Void)

- **Purpose:** Updates the postsCreated field for a user
- **Parameters:**
 - **uid:** The user's document ID
 - **postsCreated:** The new postsCreated count
 - **completion:** Callback returning success/failure
- **Process:**
 - Updates the postsCreated field in the specified user document
- **Usage in CreatePostView:**
 - Not directly used, but similar functionality is implemented in `incrementPostsCreatedForCurrentUser()`

Next Steps

- Implement draft saving functionality
- Add support for multiple image uploads
- Implement a progress indicator during image upload
- Add form validation during typing rather than only on submission
- Add category selection in addition to tags

Kevin Yue

Sprint 1 Task Documentation

Overview

Set up firebase instance for user auth, post/user database, etc. Created functional navigation bar, and wrapping view around all view pages.

Components

1. **Firestore instance(setup on web firebase console)**

I created an instance of Firestore for our project use, and set up usage for it within our project

- **Swift Components:**

- GoogleServiceInfo.plist
 - config file downloaded from firebase console allowing instance of Firestore to access google services needed for our project, ie auth, get/receive data, etc

ServiceCenter_App

- Within our app root file, I created a class AppDelegate to setup/initialize firestore instance.
- Create shared reference through an environment variable passed to content view, allowing all views to access and interact with the remote firestore

2. **NavBar**

Provides a navbar present on every page, allowing for easy navigation throughout views

- **Swift Components**

- Simple navbar in contentview using TabView wrapper
 - .onAppear functionality to check if user logged in on contentView loading in: accordingly redirect user to either login page or home page view, depending on if they are logged in or not
-

Designer Notes

To interact with firestore db, follow example function I created in contentView

Next Steps

- Create functions to easier interact with firebase for specialized purposes
- Start working on sprint 2 task(profile page), as sprint 1 tasks were completed earlier than expected

Sprint 2 Task Documentation

Overview

The goal of my task in Sprint 2 was to implement a profile page, allowing users to see their personal information/user authentication information: username, full name, description, profile image, etc. The user should also be able to edit all of these fields, and see changes to their profile updated in real time.

Components

1. ProfileViewModel

The `ProfileViewModel` stores functions that interact with the firestore, specifically with regards to user profile information

Implemented Functions:

- `modifyUserProfile(field: String, newVal: String, uid: String)`
 - Modifies the user table in firestore
 - Given the uid of the user, changes the specified field value to the new value specified

Key Challenges:

- Addressing issues with asynchronous updating, especially when needing to display newly edited fields that were updated with `modifyUserProfile`

2. ProfileView

The `ProfileView` Displays the current user's profile information, along with the sign out button

Implemented Functions/Components:

- `updateProfileImage`
 - Fetches the user profile image from the current user's firestore table entry
 - Fetched data is in url format
 - Converts data to a `UIImage` type, and returns base case of empty profile picture if error occurs or user hasn't uploaded a profile image yet
- `View`
 - Main view displays user info in a `Vstack`, in order profile image, name, description
 - Gear icon in upper right corner takes you to a settings page, where you can modify fields
 - Sign out button at bottom clears current user information(effectively signing the user out) and reloads the conditional in contentview to navigate back to the login page.

Key Challenges:

- Addressing issues with asynchronous updating: profile information wasn't loading on first appear, fixed by changing updating functions to asynchronous to stop view from loading before user information was fetched
- Sign out functionality: debugging issues that were not fully signing out the user on click

3. SettingsView

The `SettingsView` was created as a "child" of `ProfileView`, to separate editing of the profile from display to clear up the ui.

Implemented Functions/Components:

- `updateProfileImage`
 - Functionally, the same as `updateProfileImage` in `ProfileView`
- `uploadProfileImage(string: uid)`
 - Uploads the `UIImage` currently in the variable `selectedImage` to the firestore.

- Takes a uid, telling the function which entry in the table “users” to update
 - Converts the image to a string url to be properly stored
- **updateProfilePI()**
 - Fetches fields such as name and description from the current user’s entry in the firestore.
 - Stores these in state Variables.
-
- **View**
 - Main view displays current user info in a vstack, in order profile image, name, description
 - Current profile image is displayed, alongside an image selector that will open up the user’s gallery to select a new profile image
 - Info for text fields(ie name) is displayed in an editable text box
 - “Save changes” buttons to save any edited changes

Key Challenges:

- Using existing functions written by other team members: had to modify functionality of some functions to work using parameters instead of state vars so I could use them in my view as well
- Async issues: User edited changes wouldn’t reflect until views were reloaded,

Next Steps

- Clean up code: remove redundant functions(ie updateProfileImage in Settings and Profile)
- Make ui more visually appealing, potentially using some alternate templates/libraries other than default swift
- Add more fields, and location detection displayed on profile

Sprint 3 Task Documentation

Overview

The goal of my task in Sprint 3 was to implement advanced filtering features, and location. This involved two aspects: hashtags and location. The hashtag features involved allowing users to add hashtags when creating posts, and allowing users to filter by hashtags in the home screen. The location features involved allowing users to choose from multiple ways of selecting a

location, allowing users to set a location in their profile, and allowing users to filter posts in the home screen by a central location and a range in miles.

Components

Location

Viewmodels/managers

LocationManager:

LocationManager handles location variables(location, region, location snapshot) and location permissions.

- **Implemented Functions:**
 - **requestLocation():**
 - Checks permission status for location sharing
 - If permission already authorized, returns current user's location
 - If not, displays the ios window for location permissions
 - **generateSnapshot(width: CGWidth):**
 - Generates a snapshot of map, centered at the current location
 - **getAreaFromCoords():**
 - Sets the neighborhood and city vars of locationManager based on the current location

LocationSearchViewModel:

LocationSearchViewModel stores info on location search results, within LocationMapView

- **Variables:**
 - results: stores an array of possible locations from a query
 - searchableText: the text that is used to query for locations
 - locSearchCompleter: instance onf MKLocalSearchCompleter, used to query IOS maps data for locations given a search query
- **Implemented Functions:**
 - **searchAddress(searchableText: String):**
 - Calls the search completer on searchableText, starting the process of querying for possible addresses
 - **completerDidUpdateResults(completer: MKLocalSearchCompleter):**

- Takes in completer that is returned from the previous function.
- On completion and success, updates the results array with parsed address results
- **completer(_ completer: MKLocalSearchCompleter, didFailWithError error: Error):**
 - Takes in the completer
 - On error, print error

Views:

LocationSearchView:

Displays the search bar, and recommended searches in LocationMapView.

Variables:

- onRegionSelected: a closure that returns the selected location option's region
- locationSearchViewModel: instance of locationSearchViewModel that stores addresses, and handles querying for potential addresses from a search tet

Functions:

- **getPlace(from address: AddressResult):**
 - Given a search result, parse it into a readable format that can be displayed in the view
 - Returns the region of the result(so that the map can be centered on it)

View format:

- Textfield(to type in search query)
- List(Displays possible suggestion based on query)

LocationMapView:

Represents the location interaction component as a whole: centered around an interactive map, which can be dragged, or centered on searched location/current user location. Returns a closure containing the current location position selected.

Variables:

- locationManager: Instance of locationManager that allows location info to be stored and passed in/out of this view
- onLocationSaved: closure that returns location
- region/cameraPosition: determines what part of the map is showing
- curCoords: current center coordinates

View format(HStack):

- Back button(returns to parent view)
- SearchBar

- Map(interactive map, with overlaid circle pointer to show user where they currently are)
- Current location button(centers map at current location)
- Current location coordinates
- Save location button(returns to parent view, returning the current location)

LocationSelectView:

Component linking to LocationMapView, allowing users to see and edit their location information(in profile, filtering, creating posts)

Variables:

- locationManager: instance of locationManager that is shared with it's parent view, and locationMapView.
- isMapViewPresent: determines whether to show locationMapView or not
- authViewModel: instance of authViewModel. Used to, on first render of LocationSelectView, set the starting location to the user's set location in their profile

View format(GeometryView, to dynamically size view to parent view, and proportion map and text sections)

- Snapshot: display snapshot of current location, clickable to send users to LocationMapView
- LocationMapView: displays locationMapView. On closure returned, update LocationManager with new location, new snapshot, and new neighborhood/city
- Display of neighborhood and city

Hashtags

Viewmodels/managers

HashtagViewModel

Stores hashtag information and add/remove functionality

Variables:

- selectedHashTags: list of currently selected hashTags
- existingHashTags: list of Hashtags that have been used
- defaultHashTags: list of default hashTags

Functions:

- addHashTag(hashTag: String): adds hashTag to selectedHashTags
- removeHashTag(hashTag: String): removes hashTag from selectedHashTags

Views:

Tagview:

Display of a single tag icon.

View format(Hstack):

- Text(tag text)
- Remove marker(x): removes the tag from HashTagViewModel

FlexibleView:

View wrapper allowing tags to dynamically fill up/create new rows, creating a more user friendly layout

HashTagMenuView:

View displays interface for adding/removing hashTags to a parent view.

Variables:

viewModel: instance of hashTagViewModel to store selected hashtags, add/remove hashtags

curHashTag: the current string in the search box

filteredSuggestions: list of suggestions displayed based on the curHashTag: filters on whether the suggestion contains curHashTag as a substring(case insensitive)

View format(ScrollView):

- FlexibleStack: wrapping around the list of hashtags
 - TagView: for each hashtag in viewModel.selectedHashtags, display as a tagview
 - Searchbar: TextField allowing users to type in a hashtag
 - Suggestions list: dropdown list of suggestions below searchBar, displaying filteredSuggestions
-

Filters

Views:

LocationFilterView:

View displays what user sees on clicking location in the filter menu.

Variables:

locationManager: binds to the parent(filterMenuView) and the locationSelectView within this view

Range: Binding variable to the parent letting user adjust the range

View format(VStack):

- LocationSelectView
- Slider(for range)

FilterSideMenuView:

View displays side menu for filtering in HomeScreenView

Variables:

isShowing: determines whether the menu is showing

filteringHashTags: the hashtags that should be applied to the search

filteringLocation: Location that should be applied to the search

filteringRange: Range that should be applied to the search

firebaseManager:

hashtagViewModel: instance of hashtagViewModel that's used to store the currently selected hashtags

locationManager: instance of locationManager used to store the current location information

FilterTypes: a enum that is used to display filtering options

selectedFilterType: the currently selected filter type from the enum

Subview

FilterSelectionOptions: the main display. Displays each of the filtering options as a clickable button, along with a button at the bottom to apply all filters to the search.

View format(VStack):

- If hashtag is the currently selectedFilterType: Display HashTagMenuView
 - If location is the currently selectedFilterType: display LocationFilterView
-

Sprint 1 Task Documentation

Overview

The goal of my task in Sprint 1 was to implement core user authentication functionality, allowing users to register and log in to the app. This included developing the `RegistrationView`, `LoginView`, and the underlying `AuthViewModel` to handle Firebase authentication.

Components

1. `AuthViewModel`

The `AuthViewModel` serves as the primary logic layer for user authentication. It connects the views (`LoginView` and `RegistrationView`) to Firebase for handling registration, login, and error management.

- **Implemented Functions:**

- `register(email:password:fullname:)`:
 - Creates a new user in Firebase Authentication.
 - Stores additional user details (full name and email) in Firestore.
 - Sets the user session upon successful registration.
 - Handles errors gracefully and updates the view with error messages.
- `authenticate(email:password:)`:
 - Authenticates a user using Firebase Authentication.
 - Sets the user session upon successful login.
 - Displays error messages in case of failures.
- `fetchCurrentUser()`:
 - Fetches the logged-in user's data from Firestore to update the session.

- **Key Challenges:**

- Managing async code for Firebase operations.
 - Resolving issues with `@Sendable` closures and avoiding memory capture problems.
 - Handling the navigation state after registration.
-

2. `LoginView`

- Provides a simple UI for users to log in to the app.
 - **Features Added:**
 - Input fields for email and password.
 - A "Login" button to trigger the `authenticate()` method in `AuthViewModel`.
 - Error message display for invalid credentials or failed authentication.
 - A navigation link for users to move to the `RegistrationView` if they don't have an account. ("Don't have an account? Create one")
 - **Challenges Addressed:**
 - Ensuring the "Don't have an account? Create one" section is visually aligned.
 - Integrating `AuthViewModel` via `@EnvironmentObject` for smoother state management.
-

3. `RegistrationView`

- Provides the interface for users to create a new account.
 - **Features Added:**
 - Input fields for full name, email, and password.
 - A "Sign Up" button to trigger the `register()` method in `AuthViewModel`.
 - Error message display for invalid inputs or failed registration.
 - Navigation logic to proceed to the next screen upon successful registration.
 - **Challenges Addressed:**
 - Using `NavigationView` for seamless navigation between `LoginView` and `RegistrationView`.
 - Ensuring the UI transitions are smooth and error messages are correctly displayed.
-

Summary of Challenges

1. **Firestore Integration:**
 - Connecting with Firebase Authentication and Firestore for managing user data.
 - Resolving malformed or expired credential errors during login attempts.
2. **State Management:**
 - Using `@State`, `@Published`, and `@EnvironmentObject` effectively to propagate changes between views and the view model.

3. UI Design:

- Creating intuitive and responsive views for login and registration.
 - Aligning the "Create one" link correctly under the login button.
-

Next Steps

- Integrate further features, such as password reset or profile editing.
 - Add navigation to the app's main homepage upon successful login.
 - Add username feature for adding friends and easier lookup (backlog?)
-

Sprint 2 Task Documentation

Overview

The goal of my task in Sprint 2 was to implement a **unique username feature** in the authentication system. This allows users to register with a **custom username** and use it to log in instead of just their email. The system ensures that **each username is unique** by checking it against stored usernames in **Firestore** before allowing registration.

Additionally, a **Forgot Password feature** was introduced, enabling users to reset their passwords and recover their accounts more easily.

Components

1. AuthViewModel

The `AuthViewModel` manages authentication logic and Firestore interactions. It now includes additional logic for:

Handling unique usernames

Allowing login with either username or email

Resetting passwords via email

Implemented Functions:

- `register(username: email: password: fullname:)`
 - Registers a new user with Firebase Authentication.

- **Before registering**, it checks if the username is already taken by querying Firestore.
- If the username is available, the user is created in Firebase, and their details (including the username) are stored in Firestore.
- If the username is already taken, an **error message** is displayed, and registration does not proceed.
- **authenticate(usernameOrEmail: password:)**
 - Allows login using **either** a username **or** an email.
 - If the input contains '@', it is treated as an **email** and directly used for Firebase Authentication.
 - Otherwise, it looks up the **corresponding email** using the username in Firestore and then authenticates using that email.
 - Displays an **appropriate error message** if login fails.
- **checkIfUsernameExists(username: completion:)**
 - Queries Firestore to check if the entered username **is already in use**.
 - If the username exists, `usernameTaken = true`, and **registration is blocked**.
 - If the username is available, **registration proceeds normally**.
- **fetchEmailFromUsername(username: completion:)**
 - Retrieves the **email address** associated with a username from Firestore.
 - This enables logging in with a **username**, as Firebase Authentication **only supports email-based login** natively.
- **saveUserToFirestore(uid: username: fullname: email:)**
 - Saves the newly registered user's **UID, username, full name, and email** to Firestore.
 - Ensures that **each user has a unique username** stored for future lookups.

Key Challenges:

- Ensuring that **username uniqueness** is enforced correctly before registration.
- Allowing **both username and email login** while keeping Firebase Authentication's email-based system intact.
- Handling **asynchronous Firestore queries efficiently** to minimize delays in username verification.

2. RegistrationView

The **RegistrationView** UI was updated to:

Include a username field

Validate username uniqueness before allowing registration

Sync with LoginView for a consistent experience

Features Added:

- Input fields for **Full Name, Username, Email, and Password**.
- When the user enters a username and presses **Sign Up**, the system:
 - Checks whether the **username is already taken**.
 - Displays an **error message** if the username is unavailable.
 - Proceeds with registration **only if the username is unique**.
- Navigation logic ensures the user **only moves forward after successful registration**.

Challenges Addressed:

- **Preventing duplicate usernames** at the UI level instead of just in Firestore.
 - **Displaying real-time feedback** when a username is already in use.
-

3. LoginView

The **LoginView** was modified to support logging in with **either a username or email**.

Additionally, a **Forgot Password feature was introduced** to improve user account recovery.

Features Added:

- The first input field now says **"Username or Email"** instead of just "Email".
- The **login button** now determines whether the entered value is an **email or a username**.
 - If it's a **username**, the system **retrieves the associated email** from Firestore before attempting authentication.
 - If it's an **email**, authentication proceeds as usual.
- Added **Forgot Password functionality**, allowing users to reset their password via email.

Challenges Addressed:

- Removing unnecessary **username validation** in **LoginView**.
 - Ensuring **smooth login** when either a username or an email is provided.
 - **Syncing LoginView and RegistrationView** to work seamlessly together.
-

4. ForgotPasswordView (NEW FEATURE)

The **Forgot Password** screen allows users to reset their passwords via email.

Features Added:

Users enter their email and receive a password reset link.

Displays error messages if an invalid email is entered.

Syncs with LoginView for a smooth recovery process.

Challenges Addressed:

- Making sure **only valid email addresses** can receive reset links.
 - Preventing **error spam** when users enter invalid emails multiple times.
-

User Story: Password Recovery & Login Fixes

As a forgetful user, if I change my login credentials, I can sign back in. (6 points)

Tasks Completed:

1 Navigate logout directly to the login page (3 hours)

2 Ensure **LoginView** and **RegistrationView** are in sync with changes made (3 hours)

3 Added **ForgotPasswordView** so users can recover accounts more easily.

Next Steps

- Implement **profile editing** so users can change their username if needed.
 - Allow users to **reset passwords using their username** instead of email.
 - Add the ability to **search for users by username** to enable **friend requests or messaging**.
 - Improve **error messages** for a more user-friendly experience.
-

Sprint 3 Task Documentation

Overview

The goal of Sprint 3 was to **enhance the user profile** by adding a **skills section** for service providers. This allows users to showcase their expertise, making it easier for clients to find the right service provider.

Additionally, a **button under the profile view** was connected to the skills display, ensuring a seamless user experience.

Components

1. ProfileView (Updated)

The **ProfileView** now includes a **Skills Section** where users can see their listed skills.

Features Added:

A **dedicated skills section** displaying the user's skills.
If the user has no skills, a **placeholder message** is shown.
Styling adjustments to ensure skills are clearly visible.

Challenges Addressed:

- Ensuring the skills section **only appears for service providers**.
 - Making the UI **dynamic**, so it updates as users add/remove skills.
-

2. Skills Section (New Component in ProfileView)

A new section was introduced to display **user skills** retrieved from Firestore.

Task 1: Add Skills Section to Profile (2 hours)

Added a **skills display** under the profile details.
Connected the skills list to **Firestore**, ensuring it **auto-updates**.
Implemented **error handling** if skills fail to load.

Implementation:

- Fetches the **skills** field from the **user's Firestore profile**.
- If skills exist, they are displayed in a **scrollable list**.
- If no skills exist, users see **"No skills added yet."**

Example UI:

```

if let skills = authViewModel.currentUser?.skills, !skills.isEmpty {
    VStack(alignment: .leading) {
        Text("Skills")
            .font(.headline)
            .foregroundColor(.white)

        ForEach(skills, id: \.self) { skill in
            Text(skill)
                .padding(8)
                .background(Color.blue.opacity(0.2))
                .cornerRadius(8)
        }
    }
} else {
    Text("No skills added yet.")
        .foregroundColor(.gray)
}

```

3. Profile Button (Now Linked to Skills Display)

The existing **"View Skills" button** under `ProfileView` was **connected** to the actual skills section.

Task 2: Connect Button to Skills Display (2 hours)

Clicking the **"View Skills" button** now **navigates to the skills section**.

Ensured **smooth transitions** between profile details and the skills section.

If a user has no skills, they are **prompted to add skills**.

Implementation:

- Updated the **button action** to navigate to `SkillsView`.
- Ensured **Firestore syncs** so skills are always up to date.

Updated Button Code:

```

Button(action: {
    showSkills.toggle()
}) {
    Text("View Skills")
        .padding()
        .frame(maxWidth: .infinity)
        .background(Color.blue)
        .foregroundColor(.white)
}

```

```
        .cornerRadius(10)
    }
    .sheet(isPresented: $showSkills) {
        SkillsView()
    }
}
```

Challenges Addressed:

- Ensuring the **button properly updates** when skills change.
 - Preventing **duplicate screens** when navigating between profile and skills.
-

User Story: Profile with Skills Section

As a service provider, I want a profile with a skills section (4 points)

Tasks Completed:

- 1Add skills section to ProfileView (2 hours)
 - 2Connect profile button to skills display (2 hours)
-

Next Steps

- Allow users to **edit their skills** directly from their profile.
 - Enable **clients to search for service providers by skill**.
 - Add an **"Endorse Skills"** feature so users can vouch for each other.
 - Improve **profile customization** with more sections.
-

Sprint 4 Task Documentation

Alan Lam

Sprint 1 Task Documentation

Overview

Created foundation for an email notification system.

Components

1. `emails.swift`

This file houses customizable email templates as well as the logic for sending emails.

- **Implemented Functions:**
 - `sendEmail(to email:subject:content:):`
 - Purpose: Send email notifications
 - Parameters: email - the recipient subject - subject of email content - body of email
 - Usage: When the function `sendEmail()` is called with the appropriate parameters, an email will be sent to the specified user.
 - `sendLoginAttemptEmail(to email:):`
 - Modifiable email template
 - `sendRegistrationEmail(to email:fullname:):`
 - Modifiable email template
-

Designer Notes

For testing purposes, it is recommended to change the 'email' parameter to a test email you own to avoid emailing 3rd parties. Includes error handling printed to the console. Does not support @ucsc.edu emails.

Next Steps

- Add ability to send images through email

Sprint 2 Task Documentation

Overview

Made posts intractable and display all a user's posts in their profile.

Components

1. `ProfileView.swift`

A user's own profile.

- **Features Added**
 - Horizontal posts displaying only posts created by that user
 - Added placeholder profile picture if a user has not set one

2. `OtherUserProfileView.swift`

What a user sees when viewing someone else's profile.

- **Functionality**
 - Works the same as ProfileView but without access to SettingsView or sign out

3. `DetailedPostView.swift`

Adds advanced functionality to posts. Accessed by clicking on a postcard.

- **Functionality**
 - Allows navigation to post owner's profile
 - Controls the jobStatus of a post
 - Allows only the post owner to delete the post
 - Option to view requests to take job for post owner
 - Option to send requests for a job for non-post owners

4. `PostCard.swift/PostCardData.swift/PostListView.swift`

Added jobStatus to postcard

- **Functionality**
 - Header of postcard now displays current state of a post
 - Open, In Progress, Accepted, Completed
 - Isolated post navigation from PostListView into each individual postcard

5. `RequestsView.swift`

Lists users who are asking to take a job for a particular post.

- **Functionality**

- Displays username with an accept or reject button
 - If a user is accepted, jobStatus becomes “Accepted”, and their button grays out
 - If a user is accepted but then rejected, jobStatus returns to “Open”
 - Rejecting a user will remove them from the list
 - Able to click on username to navigate to requester’s profile
 - Usernames are truncated to avoid overflowing into the accept button
 - Once a user has been accepted, there is no way to reject that user if you return to the parent view (DetailedPostView)
-

Designer Notes

There is currently a desync with the jobStatus in PostListView and ProfileView, editing jobStatus in one view is not changing the postcard display in the other.

Next Steps

- Fix jobStatus desync

Sprint 3 Task Documentation

Overview

Fixed jobStatus desync, added a rating system.

Components

1. JobStatusManager.swift

Isolated control for jobStatus

- **Features Added**
 - Changes the current job status to whatever status is passed in
 - No longer necessary, can instead update jobStatus directly in Firebase

2. PostListView.swift

- **Features Added**
 - Fixed postcards being different sizes

3. `FireBasePostManager.swift`

- **Functions Added**

- `refreshPosts()` queries firebase for any changes made to posts and refreshes data

4. `DetailedPostView.swift`

- **Features Added**

- Gave job requester ability to cancel job requests in `DetailedPostView`
- Gave accepted user for a post ability to cancel job in progress, or mark it as complete, `jobStatus` updates appropriately
- Post owner can now rate the person who marked their job as complete

5. `RatingServiceView.swift`

- **Features Added**

- Allows post owner to give a rating to user who finished their job
- Stars (1-5), Review Title, description
- When submitting a review, they are shown a confirmation view, and their post is marked as "Closed"
- Title and description are mandatory and will show an error if a review is sent without them

6. `AuthManager.swift`

- **Functions Added**

- `fetchUserByUsername()` takes a username and fetches that user object

7. `UserRatingDebugView.swift`

A debug file that allows admins to inspect and delete a user's ratings.

- **Features Added**

- Text box to submit username of user's ratings you wish to view
- Print button to print ratings to console
- Button to delete imputed user's ratings
- Button to view all reviews for this user

8. `LimitedReviewsView.swift`

Displays a subset of a user's ratings, up to two. Used to avoid cluttering in `ProfileView`

9. `UserReviewsView.swift`

Displays all of a user's reviews.

10. ReviewCardView.swift

Controls the layout of a review card. Used by UserReviewsView and LimitedReviewsView

- **Features Added**

- Text box to submit username of user's ratings you wish to view
- Print button to print ratings to console
- Button to delete imputed user's ratings
- Button to view all reviews for this user

11. Miscellaneous

- Added profile image to OtherUserProfileView
- Added signout to side bar
- Gave job requester ability to cancel job requests in DetailedPostView

Sprint 4 Task Documentation

Overview

UI updates to ProfileView and RequestsView. Integrated email notification system.

Components

1. UserPostsHorizontalView.swift

Displays a user's posts horizontally with the top post as a bigger postcard.

- **Features Added**

- Large postcard displays title, price, and days since posted
- Remaining post are smaller under the large postcard and scroll horizontally

2. PostDebugView.swift

Debug to manage posts.

- **Features Added**

- Button that deletes post for all users
- Button that resets postsCreated and reviewsCreated count for current user only
- Can navigate to UserRatingDebugView

3. Email Notifications

- **Features Added**

- Email now sent to post owner when their post is marked as complete

- Email now sent to user if they have been accepted for the job

4. DetailedPostView

Removed hard coded “date created” field. Now displays days since the post was created. Will say “Today” if created on the same calendar day. And “Xd ago” for any other.

5. DetailedPostView

Replaced static count for “post created” and “reviews given” with live tracking.

Technical Details

Core Features

[List of implemented core functionalities]

UI Components

- Custom UI Elements
- Reusable Views
- Storyboard Organization

Data Models

[Document key data models and their relationships]

Third-Party Libraries

[List of CocoaPods/SPM dependencies used]

Known Issues

[Document any known bugs or limitations]

Build Information

- Minimum iOS Version:
- Swift Version:
- Xcode Version:
- Key Configuration Files:

Testing Notes

[Document any specific testing requirements for features]